

LEMONS: Leveraging Model-Based Techniques to Enable Non-Intrusive Semantic Enrichment in Wireless Sensor Networks

Jan Novacek^{*†}, Arthur Kühlwein, Sebastian Reiter^{*},
Alexander Viehl^{*}, Oliver Bringmann^{*†}, Wolfgang Rosenstiel^{*†}

^{*}FZI Research Center for Information Technology

Haid-und-Neu-Str. 10-14, 76131 Karlsruhe

[†]University of Tübingen

Sand 14, 72076 Tübingen

Abstract—The paper presents an efficient approach to the semantic enrichment of measured sensor data in Wireless Sensor Networks (WSNs), by bridging techniques from Model-driven Software Development (MDSD) and Semantic Web Technology (SWT). Our approach reinforces data interoperability, fostering data sharing and reuse, by utilizing SWT. Model-based and type-agnostic configuration reduces the overall effort for WSN setup and maintenance, which are traditionally complex and time-consuming tasks. The presented approach addresses the problem of large-scale WSN management through the application of SWT in WSN configuration and management without requiring expert knowledge. Additionally, we present a generic architecture and an implementation which is also supplemented by hands-on descriptions of an illustrative use case. Our experimental results demonstrate that our model-based approach provides non-intrusive semantic enrichment with sub-millisecond computational overhead, as well as partially automated configuration of WSNs.

Index Terms—Internet of Things, Semantic Web of Things, Wireless Sensor Networks, Knowledge-based Engineering, Model-Driven Software Development, DevOps

I. INTRODUCTION

The Internet of Things (IoT) paradigm promises the interconnection of virtual and physical objects, with applications bringing considerable benefits to industry, healthcare, and other domains. It is estimated that currently 1.5 trillion “things” exist, of which about 99 % are projected to likely become part of a network [1]. Wireless Sensor Networks (WSNs) as a key IoT technology are increasingly applied in a wider range, with estimations putting the global WSN market at \$1.8 billion by 2024 [2]. To establish a WSN, interoperability needs to be facilitated between the network devices which typically exhibit heterogeneity in formats, domains, and types.

This heterogeneity makes data provisioning and data interpretability a major challenge [3]. Furthermore, the management of WSN applications has been reported to be a key issue [4, 5]. For large-scale WSNs with thousands of heterogeneous nodes, network configuration becomes challenging and can be a time-consuming, complex, and error-prone task [6]. The application of semantic technologies to the IoT addresses the former issue by enabling device interoperability and facilitating interpretability of sensor data [3]. In particular, ontological representations of WSNs can aid in the provisioning of measurement data, for instance by providing descriptions with explicit semantics. However, the creation of corresponding ontology instances is also affected by the aforementioned scalability issues and requires the knowledge of a domain expert. In addition to that, WSN configuration is highly dependent on application objectives and performance requirements [7]. Consequently, situation-specific considerations are required for WSN configuration. Furthermore,

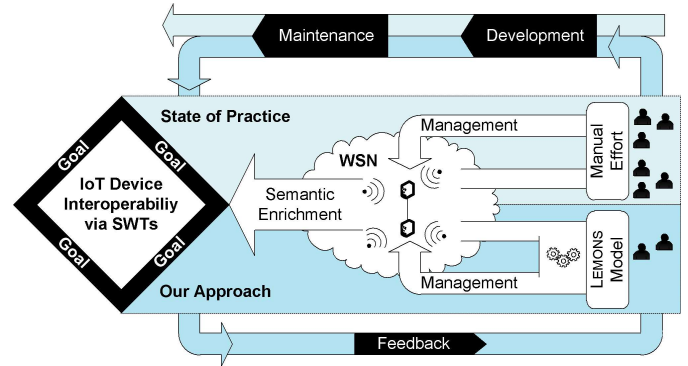


Fig. 1: Comparison of the state of practice and our approach

there exist WSN applications which require continuous adaption, for instance in construction [7].

To address the mentioned issues we introduce LEMONS¹, an approach leveraging techniques from Model-driven Software Development (MDSD) and Semantic Web Technology (SWT) to reduce complexity in development and maintenance processes and improve overall quality. In particular, this paper presents a model-based approach to improve the efficiency of WSN management and the enhancement of sensor data with explicit semantics. An inter-winning combination of model-based code generation for WSN gateways and the consequent use of standardized communication protocols, data formats and Web Ontology Language (OWL) ontologies reduces complexity in development and maintenance processes and improves overall quality by generating 100 % of LEMONS’ application code. The approach bridges the MDSD and SWT techniques by providing abstractions of ontological constructs in the metamodel. It is applicable to any kind of WSN independent of the protocols and devices used and is highly suitable for the integration into DevOps-oriented [8] processes as WSN management capabilities enabled by our approach can be used in development, setup and maintenance phases as well. Moreover, feedback from operation can directly be considered in further development. Fig. 1 outlines the envisioned approach.

The main contribution of this paper is the application of SWT in large-scale WSN configuration and management without requiring expert knowledge to address the problem of large-scale WSN application management.

¹Leveraging Model-Based Techniques to Enable Non-Intrusive Semantic Enrichment in Wireless Sensor Networks

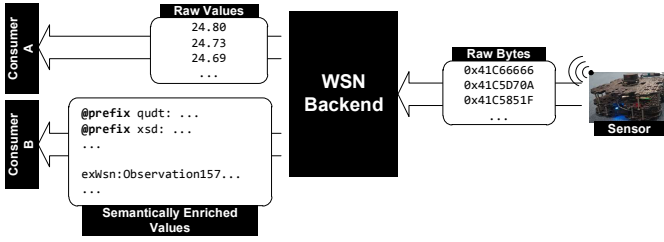


Fig. 2: Overview of the semantic enrichment process for our use case

The primary benefits of our approach are:

- | | |
|------------------|--|
| Efficiency | – Reduction of WSN setup effort and complexity |
| | – Acceleration of the WSN deployment |
| | – Easy extension and adjustment of WSNs |
| | – Development and maintenance integration |
| Quality | – Reduced setup and deployment error-proneness |
| | – Better WSN maintainability and reusability |
| Interoperability | – Improvement of measurement data interpretability |
| | – Applicability to a board range of WSNs |

We demonstrate and evaluate our proposed approach in a use case, where we enrich sensor network measurements with semantic information, see Fig. 2. In this use case, a mobile robot platform was equipped with a wireless sensor node measuring various features via different sensors. The WSN backend was processing received raw bytes from the sensor node and provisioned measurement data as raw values as well as a semantically enriched variant.

The remainder of this paper is structured as follows: Section II provides relevant information on the background and related work. Section III describes the approach while Sections IV and V, the use case and evaluation results are presented, respectively. Finally, a conclusion drawn from the experiences made while carrying out this research is given in Section VI.

II. BACKGROUND AND RELATED WORK

This section provides brief introductions to fundamental technologies namely WSNs, the Semantic Web of Things (SWoT) and MDSD. In addition to that, we compare our approach to related work.

A. Wireless Sensor Networks

A WSN is an ad-hoc network consisting of sensor nodes and at least one so-called *sink* acting as a data consumer or a bridge to another network [9, 10]. In contrast to other networks, WSNs are designed for specific applications which may be a reason that application management is an issue in WSNs [4]. The range of WSN applications is broad, including environmental, health, smart home and industrial respectively commercial applications [9].

A related approach is presented by Ibrahim and Moselhi in [7]. This approach has a focus on the design and configuration of WSN hardware and software in the context of construction. The approach proposes an iterative methodology which includes rapid-prototyping. The resulting system is self-adaptive with regard to contextual relevance monitoring. While this approach also focuses on WSN configuration, the solution concept is fundamentally different to our approach as it does neither consider MDSD nor SWT techniques and is also not concerned with the data provisioning aspect.

B. Semantic Web of Things

The SWoT is focused on providing interoperability among the heterogeneous "things" in the IoT [11]. Typical applications include Smart Homes, E-Health and Smart Cities [3]. There is a large body

of research concerned with the creation of ontologies for semantic interoperability [12, 13, 14]. With regard to ontologies in WSNs the Semantic Sensor Network (SSN) ontology plays a key role [15]. This ontology is used to model sensors, the data they produce as well as actuators. Moreover, it allows the description of used procedures for measurements and generating samples and the description of features of interest. It has a modular design and consists of the core ontology named Sensor, Observation, Sample, and Actuator (SOSA) and the upper SSN ontology. Because of its popularity, availability and standardization we also decided to reuse the SSN ontology.

Bermudez et al. recently presented in [16] *IoT-Lite*, which is also an SSN-based ontology. The authors follow the principle that IoT ontologies need to be lightweight due to the typical resource limitations of IoT devices and the demand for low processing latencies. Also, the authors refer to a work by Kolozali et al. in [17] which presents a framework for real-time semantic annotation of IoT data. While being similar to our approach, this framework builds upon Advanced Message Queuing Protocol (AMPQ) and is therefore dependent on a certain protocol. Another related work is the proposal of the Micro-Ontology Context-Aware Protocol (MOCAP) by Sahlmann and Schwotzer in [18] which introduced the idea of using micro-ontologies in the communication between sensors and other devices. While we also use micro-ontologies in our approach, we instead employ them on the layers above the individual sensors in the network hierarchy.

C. Model-driven Software Development

Model-driven Software Development (MDSD) is a technique for developing complex software by using different models which provide suitable abstractions over particular aspects of the software that is being developed. Besides analysis and optimization, these models can also be used to partially or fully generate the software code.

MDSD has been used successfully in a number of different domains [19], and the body of existing graphical and textual modeling languages is comprehensive, ranging from general-purpose modeling languages to Domain-specific Modeling Languages (DSMLs). In the case of textual DSMLs, we refer to them as Domain-specific Languages (DSLs). While graphical and textual representations form the concrete syntax of a modeling language, the metamodel to which it conforms is its abstract syntax.

The Eclipse Modeling Framework (EMF) [20] is an Eclipse-based framework that enables the development of modeling languages conforming to Ecore, which is the meta-metamodel provided by EMF. That is, Ecore provides the basic constituents from which the metamodel of a modeling language can be constructed. A large number of mature frameworks and robust tools have been developed around the EMF that allow for the rapid development of graphical and textual modeling languages, including the corresponding editors.

In research, there has been some interest in the application of MDSD to WSNs. For instance, Beckmann and Thoss [21] propose an MDSD approach leveraging the Data Distribution Service for Real-time Systems (DDS) for the runtime and communication infrastructure to expedite the software development process. Approaches such as [22], [23], and [24] propose the utilization of custom modeling languages to address the development, management, analysis, and simulation of WSNs. However, none of these works directly address semantic aspects, such as the enrichment of measurement data with explicit semantics.

III. APPROACH

This section describes our approach starting with an overview. Section III-B describes the LEMONS metamodel. Section III-C elaborates

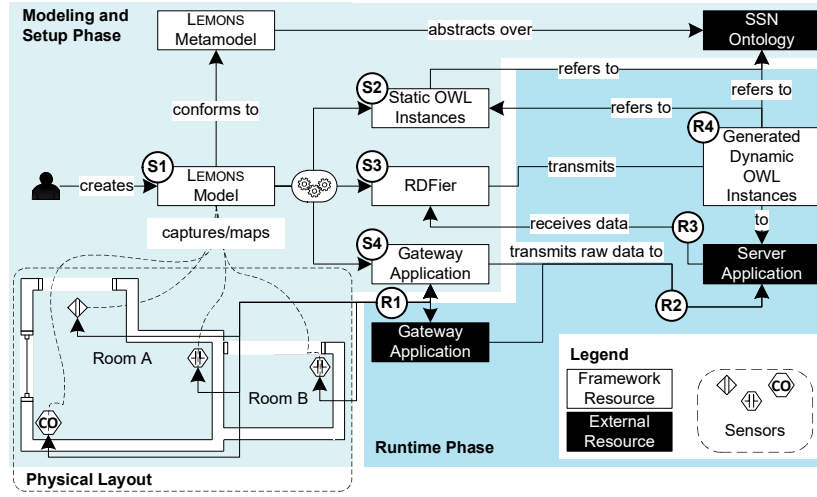


Fig. 3: Overview of the overall approach

on the the difference between *static* and *dynamic* OWL instances in semantic enrichment. Section III-D then provides a description of our proposed system architecture while Section III-E completes the description of the approach with details on the implementation.

A. Overview

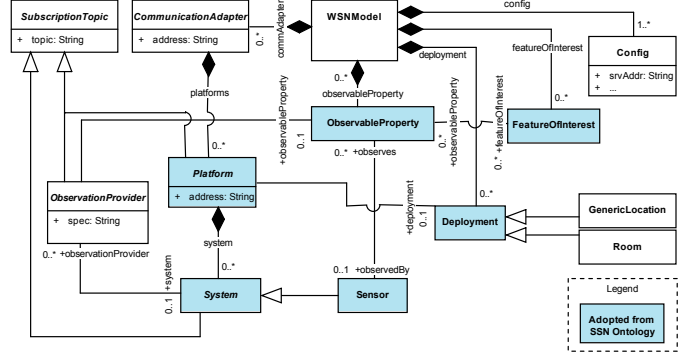
The approach is mainly structured in two phases: the *WSN modeling and setup phase* and the *runtime phase*. Please refer to Fig. 3 for an overview. During *WSN setup* a developer creates a LEMONS model of the whole WSN structure including representations of gateways and connected sensors along with the corresponding semantics (S1). This model conforms to a predefined metamodel which itself abstracts over the SSN ontology. While the creation of this model is supported by a modeling suite, it is still a manual task during which knowledge from a number of sources has to be processed. Based on the created model, three artifacts are generated: Static OWL instances describing used sensors and platforms as well as observable properties and features of interest (S2), the *RDFier* component which is responsible for the generation of dynamic OWL instances at runtime later on (S3) and optionally an executable for the WSN gateway (S4). The two preceding components are described in more detail in Section III-E.

At *WSN runtime*, the WSN gateway will receive sensor data (R1). Next, the gateway will transmit this measured sensor data to a server application (R2). The server application is responsible for the distribution of the measured sensor data and will send it to the *RDFier* component (R3) which will in turn perform semantic enrichment through the generation of dynamic OWL instances. At last, semantically enriched measurement data is transmitted back to the server (R4) from which it can be distributed and processed further.

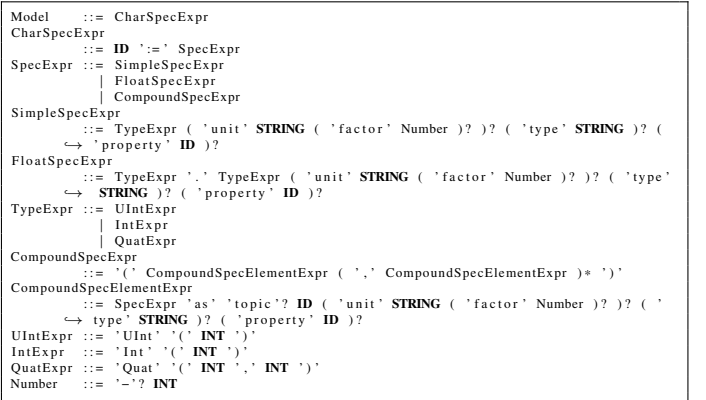
B. Metamodel

Fig. 4 (a) depicts a simplified domain diagram of the LEMONS metamodel, which provides the core concepts to model the physical and logical configuration of the WSN. In addition, the metamodel captures information required for expressing explicit semantics for the description of the WSN, including measurement data provided by the sensors.

While one of the major design goals for the metamodel was technology agnosticism in terms of modeling the WSN structure, we used state-of-the-art technologies as inspiration. For instance, the *SubscriptionTopic* element is related to the concept of topics



(a) Simplified domain diagram of the LEMONS metamodel



(b) Grammar of the sensor data interpretation DSL in W3C grammar notation

Fig. 4: Constituents of the LEMONS metamodel

in contemporary publish/subscribe messaging systems. Integration of specific technologies in the LEMONS metamodel is facilitated by specializing these core concepts. An example of this integration is given in the use cases in Section IV.

The metamodel also abstracts over key concepts of the SSN ontology with the aim of semantically enriching measurement data. The development of these abstractions cannot be easily automated due to the complexity of the SSN ontology and the differences between

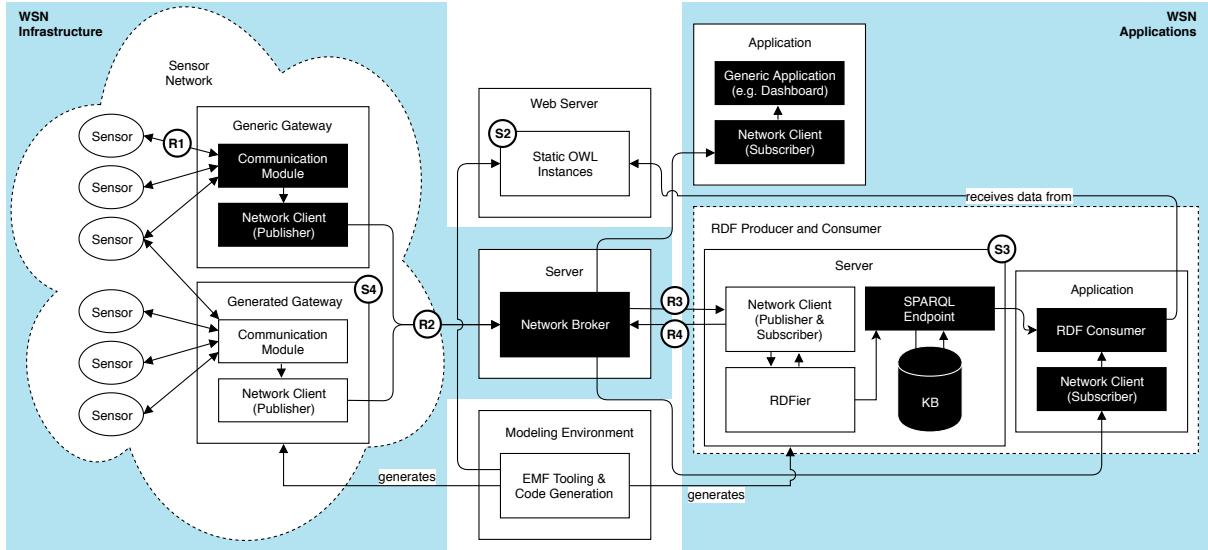


Fig. 5: Overview of the system architecture

OWL and Ecore in terms of expressiveness, as summarized in [25]. Thus, we follow a manual, iterative development strategy consisting of two phases. First, we identify and carefully analyze aspects of the SSN ontology which are central for our approach. These aspects are then subsumed into suitable abstractions, if possible, and integrated into the LEMONS metamodel. The parts of the LEMONS metamodel which have been adopted from the SSN ontology are highlighted accordingly in Fig. 4 (a).

All information about the configuration of a WSN is captured in the root-level element *WSNModel*. The *Config* element holds configuration data for connecting with the network broker.

In addition to the elements depicted in Fig. 4 (a), the metamodel comprises a textual DSL that allows for the specification of how individual bytes of raw sensor data are to be interpreted. This specification is captured in the *spec* attribute of the *Observation-Provider* element. Fig. 4 (b) depicts the grammar of the DSL in W3C grammar notation². These specifications are used by code generators to construct the correct messages for data transmission. Furthermore, the specifications assign semantics to each piece of raw measurement data in terms of the observable properties defined in a given LEMONS model.

C. Semantic Enrichment

We used the SSN ontology [26] in our approach to describe sensors, sensor platforms and measured data. With regard to measurement data units and types we used the Quantities, Units, Dimensions and Data Types (QUDT) ontologies [27]. For expressing facts related to time we used the Time ontology [28]. All knowledge with which the raw sensor measurement data is enriched stems from the LEMONS model.

In our approach we distinguish between *static* and *dynamic* OWL instances - both are OWL individuals representing relevant WSN information. When we refer to *static* OWL instances, we mean the descriptions of the WSN which do not change very frequently e.g.

connected sensor nodes and their sensors or the overall network structure. Please note, that this does not mean, that corresponding OWL instances representing such information cannot be changed over time. In contrast to that, *dynamic* OWL instances carry volatile information such as sensor measurement data. A small set of these dynamic instances forms a *micro-ontology*. Referring to SSN terms, *sensors*, *platforms*, *deployments*, *systems* and *features of interest* correspond to *static* instances in our approach, while *observations* correspond to *dynamic* instances. There are several reasons for this distinction: As static information does not change frequently it is beneficial to not transmit it over and over again thus saving bandwidth in typically resource-constrained WSNs. Besides that, when publishing static information in an ontology on a web server instead of constantly transmitting it increases the chances that it is available when needed. This also makes a complicated query mechanism unnecessary.

While the static instances are generated directly from the LEMONS model, dynamic instances are generated by the *RDFier* component, see Fig. 3. Please see List. 1 to get an impression of an excerpt of the generated static instances and the bottom listing in Fig. 10 for a generated dynamic instance. The *RDFier* component is in turn generated by the LEMONS model. In broad terms, the code generation creates the corresponding structures and expressions in the *RDFier* source code which handle each of the various sensors described in the LEMONS model. The basic capabilities of the *RDFier* comprise network connectivity to the server application, generation of *observations* from received measurement data and publication thereof via the server or to a SPARQL Protocol and Query Language (SPARQL) endpoint.

D. System Architecture

Fig. 5 gives an overview of the system architecture. The sensor network consists of multiple gateways, which are connected to sensor nodes. The gateway is responsible for facilitating communication with the connected sensor nodes and the server. The client application running on the gateway publishes the raw sensor data to the network

²<https://www.w3.org/TR/xquery-31/#EBNFNotation>

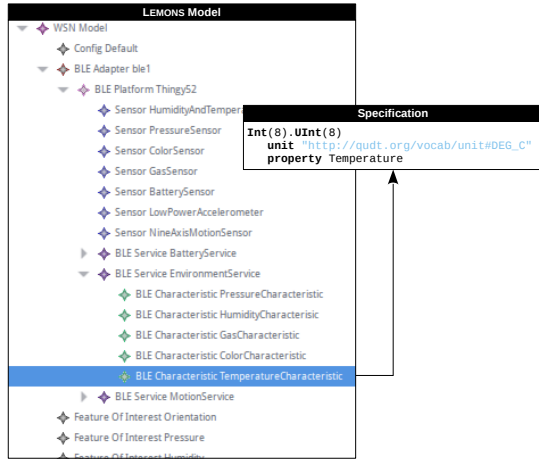


Fig. 6: EMF editor for our LEMONS model with an exemplary specification of raw temperature measurement data

broker running on the server, which is responsible for distributing the data to the applications that require it.

One of these applications is a server hosting the RDFier component, which consumes the raw sensor data and publishes semantically enriched versions of them to the network broker. In addition, these enriched data are sent to a SPARQL endpoint which is also running on the server. Consequently, applications that require enriched data have the option of subscribing to a stream of data from the network broker or querying the SPARQL endpoint.

The modeling environment contains the EMF tooling for creating and editing LEMONS models as well as the corresponding generators for the gateway application and the server hosting the RDFier and SPARQL endpoint.

Our decision to use this particular architecture is motivated by a number of considerations. Because full access to the server cannot be assumed a priori, our approach required non-intrusiveness with regard to the existing WSN. Consequently, we decided to implement the RDFier component as another publisher and subscriber running independently of the rest of the system, effectively making the RDFier an extension. In addition, placing the RDFier component inside the gateway applications would introduce communication overhead on the gateways running these applications, potentially decreasing overall network responsiveness.

E. Implementation

As shown in Fig. 3, a number of artifacts (S2) to (S4) are generated from a LEMONS model (S1), which can currently be viewed and edited using the default tree-based graphical editor that is generated by EMF from the LEMONS Ecore description. The graphical editor can be seen in Fig. 6. While the generation of the static OWL instances (S2) is handled by the Jena framework³, Epsilon Generation Language (EGL)⁴ templates are used to generate 100% of the code for the RDFier (S3) and gateway application (S4).

The gateway application is implemented within a single Scala [29] object. A basic design principle was the use of non-blocking I/O which we achieved through asynchronous stream-based processing utilizing the streaming engine Akka Streams⁵. Using Akka Streams had the advantage that we could easily define different *sources*

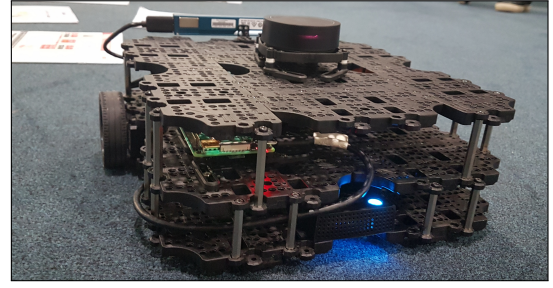


Fig. 7: TurtleBot equipped with the Nordic Thingy:52™ IoT sensor platform. The sensor platform is mounted on the lower middle in the back side of the robot

representing the sensors sending measurement data to the gateway depending on the type of the communication module. Likewise, we could define various *sinks* which represent the connections to the *network broker* back-end consuming the raw measurement data, see (R2) in Fig. 3 and Fig. 5. This pattern provides a great amount of maintainability because source and sink implementations could be reused and extended. Also, it favors the template-based code generation.

For the RDFier application, which is also implemented in Scala, we applied the same pattern as in the gateway application implementation using the Akka Streams engine. The main purpose of the RDFier component is the generation of SSN ontology *observations* as Resource Description Framework (RDF) data based on received measurement data. Therefore, the RDFier implementation has to contain code that transforms received raw measurement data into corresponding instances expressed in OWL. In a configuration for a large WSN there will be a lot of code constructs for each sensor value. This is also where the code generation comes in: Based on the specified sensors of the WSN in the LEMONS model, constructs handling each sensor value transformation are automatically generated, thus potentially saving a considerable amount of time and avoiding errors.

IV. USE CASE

This section presents an illustrative use case which has a focus on the provisioning of semantically enriched measurement data. While the presented approach is essentially applicable to any type of WSN, in our use case the realizing system is relying on MQ Telemetry Transport (MQTT) [30] as communication protocol between the Raspberry Pi 3 Model B+ gateway, generic applications and the RDFier and Bluetooth Low Energy (BLE) between the gateway and the sensor nodes.

In order to generate load- and usage-profiles we equipped the Waffle TurtleBot⁶ with an additional IoT sensor platform, the Nordic Thingy:52™, see Fig. 7. The general setup of the use case is illustrated in Fig. 2, where the backend comprises the network broker and the server (S3) in Fig. 5.

Next, we used our *modeling environment* (see Fig. 6) to generate executable code for the gateway to which the Nordic Thingy:52™ should connect for forming a WSN. In this case, we were able to generate 100% of the gateway code. In addition to that, we generated the static ontology instances according to the SSN ontology which were ought to describe the network structure and the contained sensors, see List. 1. We then operated the robot platform on both smooth and bumpy floor sections. The measurement data exhibited spikes in

³<https://jena.apache.org/>

⁴<https://www.eclipse.org/epsilon/doc/egl/>

⁵<https://doc.akka.io/docs/akka/current/stream/index.html>

⁶<https://www.turtlebot.com/>

```

<rdf:Description rdf:about="Thingy52-...">
  <sosa:hosts>NineAxisMotionSensor-...</sosa:hosts>
  <sosa:hosts>LowPowerAccelerometer-...</sosa:hosts>
  <sosa:hosts>BatterySensor-...</sosa:hosts>
  <sosa:hosts>GasSensor-...</sosa:hosts>
  <sosa:hosts>ColorSensor-...</sosa:hosts>
  <sosa:hosts>PressureSensor-...</sosa:hosts>
  <sosa:hosts>HumidityAndTemperatureSensor-...</sosa:hosts>
  <rdfs:label>Nordic Semiconductor Thingy:52 compact multi-sensor prototyping
    ↪ platform</rdfs:label>
  <rdfs:type>sosa:Platform</rdfs:type>
</rdf:Description>

```

List. 1: Definition excerpt of the sensor platform in OWL/XML

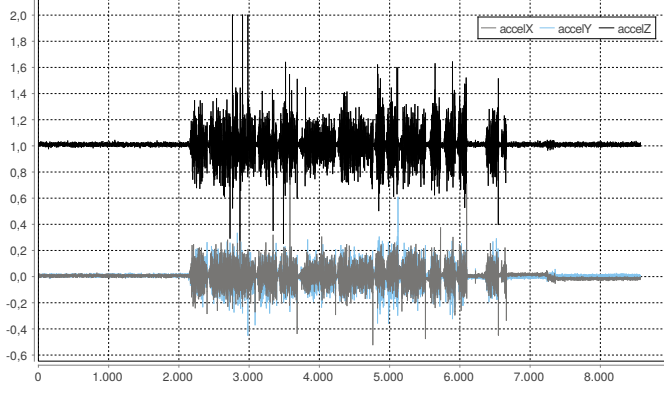


Fig. 8: Measurement data excerpt of the acceleration in $G \cdot s^{-1}$

the Z-axis of the acceleration graph when the robot operated on a bumpy floor section, see Fig. 8. To monitor the measurement we also implemented a dashboard which is capable of visualizing all captured measurement data. In addition to that all sensor measurement data was written to Comma-separated values (CSV) files and published to a MQTT broker.

While the visualizations and CSV files are appropriate when actually performing the measurements, they are not helpful when interpreting measurement data at a later point in time or when sharing the data.

This is where the RDFier component comes in: In addition to the generation of the gateway code and the static instances from the SSN ontology, we used our EMF modeling environment to generate the RDFier which is capable of producing and publishing SSN ontology instances representing sensor *observations* as RDF data. For a comparison between raw CSV data and excerpt of corresponding enriched measured data, see Fig. 10.

V. EVALUATION

In order to evaluate our approach with respect to its overall quality, we validated the correct generation of LEMONS code, as well as the proper serialization of semantically enriched measurement data. As apparent from Fig. 10, our approach enriches raw measurement data with explicit, interoperable semantics due to the utilization of standardized ontologies. Likewise, our integrated modeling environment is capable of generating not only executable gateway code, but also the static ontology and the RDFier component which are both responsible for the semantic enrichment.

Furthermore, the LEMONS model in our use case could be created in the modeling environment by a person without expert knowledge due to the suitable abstractions provided by the metamodel. These models can be stored and loaded, supporting WSN configuration and management.

As WSNs are often employed in real-time applications [31], latency is a key performance factor we considered during our evalu-

ation. To evaluate the impact of our framework on WSN latency, we conducted three different measurements on an Intel® Core™ i5-750 CPU at 2.67 GHz with 8 GB RAM. Data for the first, second, and third measurement were collected by letting the executable run with the same WSN configuration for 5, 60, and 4 minutes, collecting over 179 000, 266 000, and 17 000 data points, respectively. For each measurement, the executable was simply run until an adequate number of data points was acquired.

The goal of the first measurement was to get an insight into the execution times of the individual phases of the micro-ontology generation process, which comprise MQTT translation, ontology generation, serialization, and compression, in order. In particular, we were interested in the performance implications of using different OWL serialization formats and the effects of GZIP compression on generation time.

Fig. 9 (a) and (c) show the results of the first measurement, which imply a non-significant difference in generation times between the individual OWL serialization formats. The majority of generation is performed within 500 μs , with some outliers exceeding 1 ms, which can be most likely explained by garbage collection side effects of the Java Virtual Machine.

Consequently, we can draw the conclusion that any overhead introduced by our approach in terms of latency is going to be dominated by the transmission of the ontology instances over the network, and not by the generation of the dynamic ontology instances. This is a direct result of the considerations described in Section III-D, in which we essentially had to make a trade-off between non-intrusiveness and latency, favoring the former over the latter. It is obvious that optimal latency would be achieved by integrating the RDFier component directly on the server hosting the network broker as depicted in Fig. 5, but in this case full access to the server would be required which is not always possible.

With the second measurement, we intended to measure the effects of GZIP compression on micro-ontology size for the serialization formats used in the first measurement. The results of this measurement are shown in Fig. 9 (b). While the achievable compression ratio is highly dependent on the chosen serialization format, the inter-format variance between the compressed micro-ontology sizes is negligible. This low variance in compressed size allows for a high degree of freedom with respect to serialization format selection.

With the third measurement, we intended to analyze the micro-ontology generation time in relation to the generated micro-ontology size. Fig. 9 (d) shows the results of the measurement for each generated property type using Turtle as OWL serialization format. The micro-ontology instances generated by the RDFier all have sizes less than 1 KB and non-significant inter-property size variance, due to our provisioning of semantic information in static and dynamic form, as shown in Fig. 3 (S2) and (R4), respectively.

VI. CONCLUSION AND FUTURE WORK

We have presented a model-based approach to the configuration and generation of executable code for WSNs which includes a framework for an ontology-based semantic enrichment and provisioning of measurement data.

The current version of the WSN metamodel does not capture the entirety of the SSN ontology. On the one hand, this is due to limitations in the ontology itself. For instance the currently published set of Quantities, Units, Dimensions and Data Types (QUDT) ontologies does not yet contain definitions for the representation of specialized vectors such as those representing the attitude of a sensor platform.

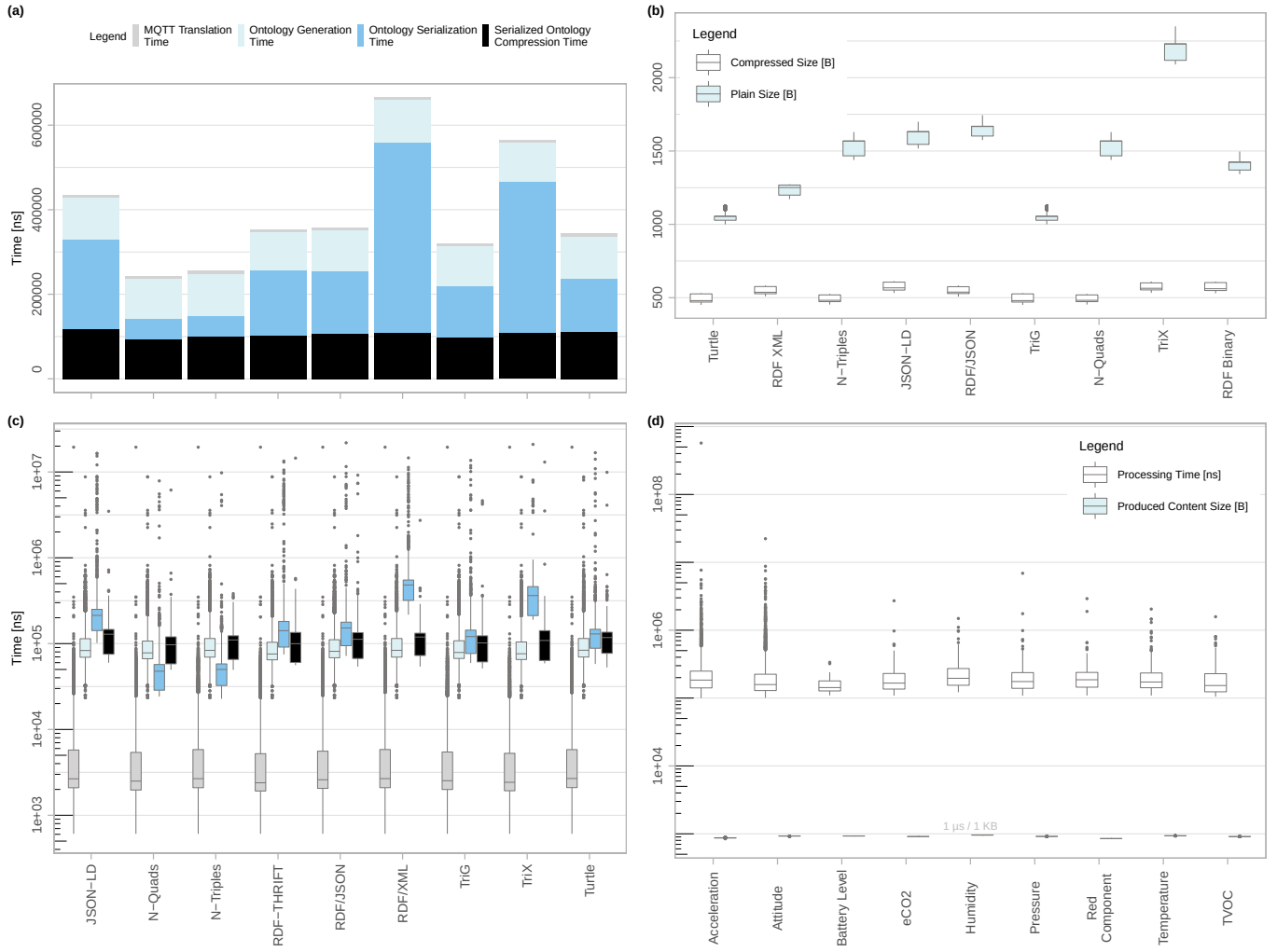


Fig. 9: (a) and (c) depict the execution times of individual micro-ontology generation phases for different OWL serialization formats using mean and exact values, respectively. (b) shows the effects of GZIP compression on micro-ontology size for the serialization formats. (d) depicts the processing time and micro-ontology size for each generated property type using Turtle as OWL serialization format

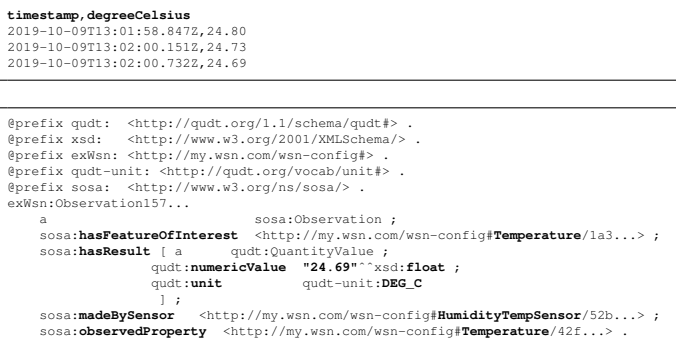


Fig. 10: CSV (top) versus enriched OWL measurement data (bottom)

On the other hand, the mapping of the SSN ontology to EMF is challenging due to differences in the expressiveness of the respective (meta-)metamodels. For example, multiple classification for class instances is difficult to represent in Ecore. Bridging this gap is non-trivial and will be a major part of future efforts. One possible solution could be the provision of generic mechanisms that allow the definition of such ontological constructs within the modeling environment. Such

an approach could potentially be generalizable to arbitrary ontologies.

While the current tree-based implementation of the modeling environment is functional, a graphical modeling environment using suitable visualizations can further improve usability. The code generation framework could be extended to allow the generation of server configuration code. For commercial off-the-shelf gateways, the sensor data interpretation DSL can be extended to support industry-standard data exchange formats such as JSON or XML. Another desirable feature would be the possibility of live network management using the corresponding LEMONS model.

Our approach paves the way for significant reductions of WSN setup effort and management complexity by generating 100 % of LEMONS' application code. In addition to that, our approach accelerates the transition to general device interoperability in IoT with regard to measurement data provisioning. Furthermore, our findings indicate a significant reduction in MQTT payload sizes by compressing the micro-ontology instances. Future work will evaluate the incorporation of advanced delta-based techniques for network traffic reduction into our approach.

ACKNOWLEDGMENT

This paper is partially funded by the BMBF within the project COMPACT (grant number 01/S17028C), and by the BMWi within the project InsightProducts (grant number 228EN/2).

REFERENCES

- [1] S. Greengard, *The internet of things*. MIT press, 2015.
- [2] P. Harrop and R. Das, “Wireless Sensor Network (WSN) 2014–2024: Forecasts, Technologies, Players,” Boston, MA, USA, 2014.
- [3] F. Shi, Q. Li, T. Zhu, and H. Ning, “A survey of data semantization in internet of things,” *Sensors*, vol. 18, no. 1, p. 313, 2018.
- [4] J. Yick, B. Mukherjee, and D. Ghosal, “Wireless sensor network survey,” *Computer networks*, vol. 52, no. 12, pp. 2292–2330, 2008.
- [5] H. Mostafaei and M. Menth, “Software-defined wireless sensor networks: A survey,” *Journal of Network and Computer Applications*, vol. 119, pp. 42–56, 2018.
- [6] S. Lee, “Reducing Complexity of Large-scale Network Configuration Management,” Ph.D. dissertation, Pittsburgh, PA, USA, 2010, aAI3415822.
- [7] M. Ibrahim and O. Moselhi, “Wireless sensor networks configurations for applications in construction,” *Procedia Engineering*, vol. 85, pp. 260–273, 2014.
- [8] D. Mala, *Integrating the Internet of Things Into Software Engineering Practices*, ser. Advances in Systems Analysis, Software Engineering, and High Performance Computing (2327-3453). IGI Global, 2019. [Online]. Available: <https://books.google.co.uk/books?id=GPGCDwAAQBAJ>
- [9] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, “Wireless sensor networks: a survey,” *Computer networks*, vol. 38, no. 4, pp. 393–422, 2002.
- [10] A. C. Djedouboum, A. Ari, A. Adamou, A. M. Gueroui, A. Mohamadou, and Z. Aliouat, “Big data collection in large-scale wireless sensor networks,” *Sensors*, vol. 18, no. 12, p. 4474, 2018.
- [11] A. J. Jara, A. C. Olivieri, Y. Bocchi, M. Jung, W. Kastner, and A. F. Skarmeta, “Semantic web of things: an analysis of the application semantics for the IoT moving towards the IoT convergence,” *International Journal of Web and Grid Services*, vol. 10, no. 2-3, pp. 244–272, 2014.
- [12] C. Schlenoff, T. Hong, C. Liu, R. Eastman, and S. Foufou, “A literature review of sensor ontologies for manufacturing applications,” in *2013 IEEE International Symposium on Robotic and Sensors Environments (ROSE)*. IEEE, 2013, pp. 96–101.
- [13] A. Gyrard, S. K. Datta, and C. Bonnet, “A survey and analysis of ontology-based software tools for semantic interoperability in IoT and WoT landscapes,” in *2018 IEEE 4th World Forum on Internet of Things (WF-IoT)*. IEEE, 2018, pp. 86–91.
- [14] S. D. Nagowah, H. B. Sta, and B. Gobin-Rahimbux, “An overview of semantic interoperability ontologies and frameworks for IoT,” in *2018 Sixth International Conference on Enterprise Systems (ES)*. IEEE, 2018, pp. 82–89.
- [15] M. Ganzha, M. Paprzycki, W. Pawlowski, P. Szmaja, and K. Wasielewska, “Semantic technologies for the IoT-an inter-IoT perspective,” in *2016 IEEE First International Conference on Internet-of-Things Design and Implementation (IoTDI)*. IEEE, 2016, pp. 271–276.
- [16] M. Bermudez-Edo, T. Elsaleh, P. Barnaghi, and K. Taylor, “IoT-Lite: a lightweight semantic model for the internet of things,” in *2016 Intl IEEE Conferences on Ubiquitous Intelligence & Computing, Advanced and Trusted Computing, Scalable Computing and Communications, Cloud and Big Data Computing, Internet of People, and Smart World Congress (UIC/ATC/Scal-*

- Com/CBDCCom/IoP/SmartWorld*). IEEE, 2016, pp. 90–97.
- [17] S. Kolozali, M. Bermudez-Edo, D. Puschmann, F. Ganz, and P. Barnaghi, “A knowledge-based approach for real-time IoT data stream annotation and processing,” in *2014 IEEE International Conference on Internet of Things (iThings), and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom)*. IEEE, 2014, pp. 215–222.
 - [18] K. Sahlmann and T. Schwotzer, “Mocap: Towards the semantic web of things,” in *SEMANTICS (Posters & Demos)*, 2015, pp. 59–62.
 - [19] A. Vallecillo, “On the Industrial Adoption of Model Driven Engineering. Is your company ready for MDE?” *International Journal of Information Systems and Software Engineering for Big Companies (IJISEBC)*, vol. 1, pp. 52–68, 12 2014.
 - [20] D. Steinberg, F. Budinsky, M. Paternostro, and E. Merks, *EMF: Eclipse Modeling Framework 2.0*, 2nd ed. Addison-Wesley Professional, 2009.
 - [21] K. Beckmann and M. Thoss, “A Model-Driven Software Development Approach Using OMG DDS for Wireless Sensor Networks,” in *Software Technologies for Embedded and Ubiquitous Systems*, S. L. Min, R. Pettit, P. Puschner, and T. Ungerer, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 95–106.
 - [22] P. Zhang and J. Wang, “Management of Intelligent Campus Wireless Sensor Networks Based on Runtime Model,” *Journal of Computer and Communications*, vol. 03, pp. 22–31, 01 2015.
 - [23] N. Ma, Y. Lin, and X. Zhou, “WiFi Sensor Network Management Based On Runtime Model,” *International Journal of Future Generation Communication and Networking*, vol. 9, pp. 269–280, 02 2016.
 - [24] I. Malavolta, L. Mostarda, H. Muccini, E. Ever, K. Dodapaneni, and O. Gemikonaklı, “A4WSN: an architecture-driven modelling platform for analysing and developing WSNs,” *Software & Systems Modeling*, 07 2018.
 - [25] G. Hillairet, F. Bertrand, and J. Y. Lafaye, “Bridging EMF applications and RDF data sources,” in *Proceedings of the 4th International Workshop on Semantic Web Enabled Software Engineering, SWESE*, 2008.
 - [26] M. Compton, P. Barnaghi, L. Bermudez, R. García-Castro, O. Corcho, S. Cox, J. Graybeal, M. Hauswirth, C. Henson, A. Herzog *et al.*, “The SSN ontology of the W3C semantic sensor network incubator group,” *Web semantics: science, services and agents on the World Wide Web*, vol. 17, pp. 25–32, 2012.
 - [27] R. Hodgson, P. J. Keller, J. Hodges, and J. Spivak, “QUDT-quantities, units, dimensions and data types ontologies,” *USA Available <http://qudt.org> March*, 2014.
 - [28] J. R. Hobbs and F. Pan, “Time ontology in OWL,” *W3C working draft*, vol. 27, p. 133, 2006.
 - [29] M. Odersky, P. Altherr, V. Cremet, B. Emir, S. Maneth, S. Micheloud, N. Mihaylov, M. Schinz, E. Stenman, and M. Zenger, “An overview of the Scala programming language,” *Tech. Rep.*, 2004.
 - [30] “MQTT version 3.1.1,” *Edited by Andrew Banks and Rahul Gupta*, vol. 10, 2014.
 - [31] A. Ali, Y. Ming, S. Chakraborty, and S. Iram, “A comprehensive survey on real-time applications of WSN,” *Future internet*, vol. 9, no. 4, p. 77, 2017.